

Database

Lecture 4: Basic SQL

Yunyong Ko
School of Computer Science and Engineering
Chung-Ang University

Outline

❑ Basic SQL: Data Definition Language



- Introduction to SQL
- Creating Schemas and Tables
- Specifying Constraints
- Modifying Schemas: DROP and ALTER

❑ Basic SQL: Data Manipulation Language

- Data Update: INSERT, DELETE, and UPDATE
- Data Retrieval: SELECT-FROM-WHERE Structure

Structured Query Language (SQL)

❑ What is a Structured Query Language (SQL)?

- Special-purpose *declarative* programming language
  - ❑ Designed for IBM's first DBMS and originally named **SEQUEL** (Structured English Query Language)
  - ❑ *Pronounced by '**S-Q-L**' or '**SEQUEL**'*

❑ Components of SQL

- **Data Definition Language (DDL)**
  - ❑ Providing commands for '**databases and tables**'
    - CREATE Schema and Table
    - ALTER and DROP Schema and Table
- **Data Manipulation Language (DML)**
  - ❑ Providing commands for '**tuples**' in tables
    - Retrieval: SELECT
    - Modification: INSERT, DELETE, UPDATE



Chamberlin, Donald D., and Raymond F. Boyce. "SEQUEL: A Structured English Query Language."
*in Proceedings of the ACM SIGFIDET (**SIGMOD**) Workshop on Data Description, Access and Control, 1974.*

Can AI Do SQL Well?

❑ What is NL-to-SQL (NL2SQL)?

- A technique to convert a natural language query into a structured SQL query to retrieve the desired data from a database



What is the average salary of the worst performing managers?

```
SELECT AVG(CAST(REPLACE(SUBSTR(T1.salary, 4), ',', '') AS REAL))
FROM employee AS T1 JOIN position AS T2 ON T1.positionID = T2.positionID
WHERE T1.performance = 'Poor' AND T2.positiontitle = 'Manager'
```



Employees

em_id	last_name	first_name	salary
0000	Milgrom	Milgrom	US$57,500.00
2222	Adams	Sandy	US$19,500.00
6543	Wood	Emily	US$69,000.00
… …	… …	… …	… …

❑ Challenges of NL2SQL

- Natural language ambiguity
- Schema understanding and linking
- Generalization to unseen databases
- Complex query generation

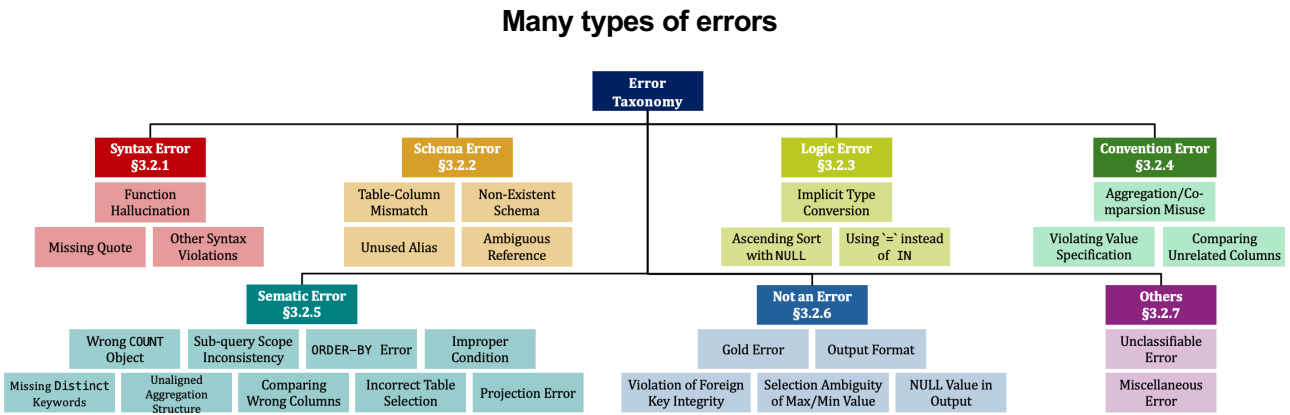
Li, Jinyang, et al. "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sqls." *NeurIPS* 2023.

Current Performance of NL-to-SQL

❑ There Still Exists the Room for Further Improvement

	Model	Code	Size	Self Consistency	Dev (%)	Test (%)
	Human Performance *Data Engineers + DB Students*					92.96
1 Mar 12, 2026	Gemini-SQL (Multitask SFT + Gemini-2.5-Pro) *Google Cloud Databases* Google Cloud Databases		UNK	*Few*	72.62	New Sub 76.63
2 Dec 4, 2025	Q-SQL *AWS-Quick Science* [Ravi Shankar et al.'25]		30B-3B-MoE	*Many*	72.99	76.47
3 July 08, 2025	Databricks RLVR 32B *Databricks Mosaic Research* Databricks Mosaic Research		32B	*Few*	-	75.68
4 Dec 15, 2025	SiriusAI-Text2SQL-32B-v2 *Tencent Data & Computation Platform Department*		32B	*Many*	72.82	75.01
5 Oct 30, 2025	Spektr-SQL *Amazon Ads-SpektrBot* [Ravi Shankar et al.]		30B-3B-MoE	*Few*	72.10	74.85

	Model	Code	Size	Oracle Knowledge	Dev (%)	Test (%)
	Human Performance *Data Engineers + DB Students*			✓		92.96
1 Dec 16, 2025	AskData + GPT-4o *AT&T CDO - DSAIR* [Shkapenyuk et al. '25]		UNK	✓	77.64	81.95
2 Sep 25, 2025	Agentar-Scale-SQL *Ant Group* [Pengfei Wang et al. '25]	[link]	UNK	✓	74.90	81.67
3 July 14, 2025	LongData-SQL *LongShine AI Research*		UNK	✓	74.32	77.53
4 Jan 02, 2026	Zhiwen-Lingsi-Agent *China Telecom, TeleAI*		UNK	✓	73.53	76.63
5 Jan 26, 2026	DeepEye-SQL *Anonymous*		UNK	✓	73.53	76.58
6 Dec 4, 2025	Q-SQL *AWS-Quick Science* [Ravi Shankar et al.'25]		30B-3B-MoE	✓	72.99	76.47



Recent LLM-based NL-to-SQL techniques are powerful, but they are far from perfect ⚠️

→ *Thus, a solid understanding of SQL is crucial!*

Li, Jinyang, et al. "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sqls." *NeurIPS* 2023.

Shen, Jiawei, et al. "A study of in-context-learning-based text-to-sql errors." *arXiv preprint arXiv:2501.09310* (2025).

Creating a Database Schema in SQL

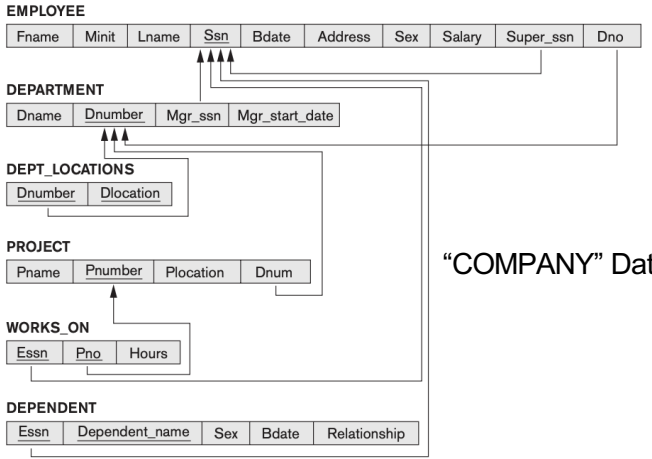
What is a Schema in SQL?

- Tables, types, constraints, views, domains, and others that describe the schema
  - Identified by a **schema name**

CREATE SCHEMA Statement

- To specify a schema name and an authorization identifier
  - Authorization identifier: to indicate the user who owns the schema

CREATE SCHEMA COMPANY **AUTHORIZATION** 'Jsmith';



"COMPANY" Database

Creating a Table in SQL

❑ CREATE TABLE Statement

- To create a *new relation* by specifying its **name**, **attributes** and **initial constraints**
  - **Table name**: for clarity, attaching the schema name to the relation name

CREATE TABLE EMPLOYEE

CREATE TABLE COMPANY.EMPLOYEE

  - **Attributes**: a list of <attribute_name, data_type> pairs
  - **Initial constraints**: attribute constraints, integrity constraints,

```
CREATE TABLE EMPLOYEE
        ( Fname          VARCHAR(15)     NOT NULL,
          Minit          CHAR,
          Lname          VARCHAR(15)     NOT NULL,
          Ssn            CHAR(9)         NOT NULL,
          Bdate          DATE,
          Address        VARCHAR(30),
          Sex            CHAR,
          Salary         DECIMAL(10,2),
          Super_ssn      CHAR(9),
          Dno            INT             NOT NULL,
        PRIMARY KEY (Ssn),
```

```
CREATE TABLE DEPARTMENT
        ( Dname          VARCHAR(15)     NOT NULL,
          Dnumber        INT             NOT NULL,
          Mgr_ssn        CHAR(9)         NOT NULL,
          Mgr_start_date DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber        INT             NOT NULL,
          Dlocation      VARCHAR(15)     NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

Specifying Attribute Constraints

❑ NOT NULL Constraint for Attributes

- Attribute values cannot be NULL
  - ❑ This is always implicitly specified for the primary key attributes (i.e., *Entity integrity constraint*)

❑ Default Values for Attributes

- To define a default value for an attribute: the clause **DEFAULT** <value>
  - ❑ The default value is included in any new tuple if *an explicit value is not provided* for that attribute

```
CREATE TABLE EMPLOYEE
        ( Fname          VARCHAR(15)     NOT NULL,
          Minit          CHAR,
          Lname          VARCHAR(15)     NOT NULL,
          Ssn            CHAR(9)         NOT NULL,
          Bdate          DATE,
          Address        VARCHAR(30),
          Sex            CHAR,
          Salary         DECIMAL(10,2),
          Super_ssn      CHAR(9),
          Dno            INT             NOT NULL,
        PRIMARY KEY (Ssn),
```

```
CREATE TABLE DEPARTMENT
    ( ... ,
      Mgr_ssn CHAR(9)          NOT NULL      DEFAULT '888665555',
      ... ,
    CONSTRAINT DEPTPK
      PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
      UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
      FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET DEFAULT     ON UPDATE CASCADE);
```

Specifying a default department for a new employee

Specifying Key Constraints

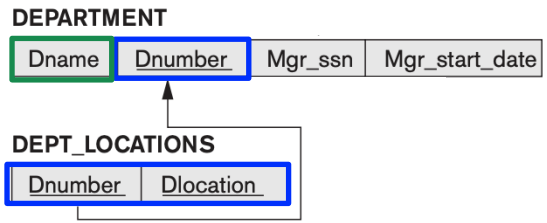
❑ PRIMARY KEY Clause

- To specify one or more attributes that make up the primary key of a relation

❑ UNIQUE Clause

- To specify alternate (unique) keys (i.e., *candidate keys*)

```
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)             NOT NULL,
          Dnumber               INT                     NOT NULL,
          Mgr_ssn               CHAR(9)                 NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT                     NOT NULL,
          Dlocation             VARCHAR(15)             NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation)
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

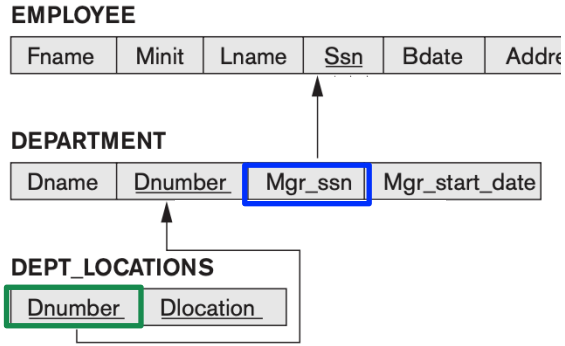


Specifying Referential Integrity Constraints

❑ FOREIGN KEY Clause

- To specify a foreign key that refers to a primary key of another relation

```
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)         NOT NULL,
          Dnumber               INT                 NOT NULL,
          Mgr_ssn               CHAR(9)             NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT                 NOT NULL,
          Dlocation             VARCHAR(15)         NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```



When the primary key attribute values are updated or deleted,
Referential integrity constraint can be violated

The default action of SQL is to ***reject*** those operations
Q. Any other way to address this issue? → **Referential Triggered Actions**

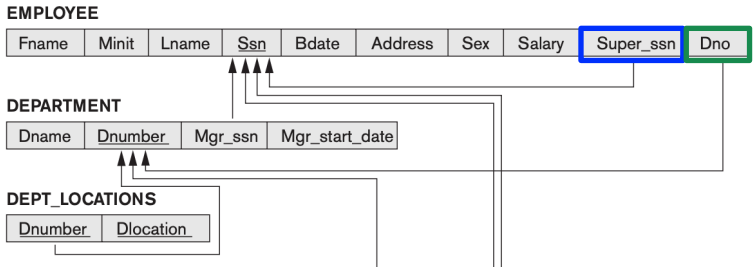
Specifying Referential Triggered Actions

❑ How to Specify Referential Triggered Actions?

- **Triggers**: ON DELETE, ON UPDATE
- **Actions**: SET NULL, CASCADE, SET DEFAULT

```
CREATE TABLE EMPLOYEE
    ( ... ,
      Dno          INT          NOT NULL        DEFAULT 1,
    CONSTRAINT EMPPK
      PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
      FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET NULL          ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
      FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                  ON DELETE SET DEFAULT       ON UPDATE CASCADE);
```

"What if the tuple referenced by others is deleted/updated?"
Setting NULL or a Default Value / Cascading a New Value to all tuples referencing the deleted/updated tuple.



EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

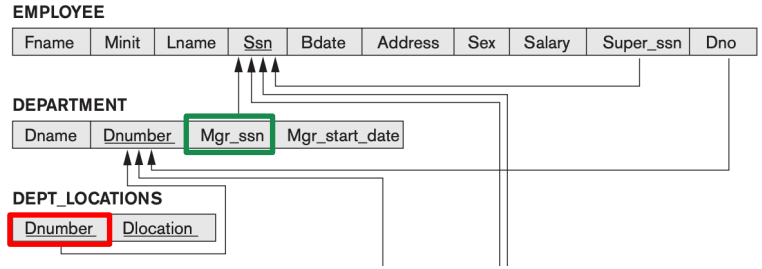
Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Specifying Referential Triggered Actions

How to Specify Referential Triggered Actions?

- **Triggers**: ON DELETE, ON UPDATE
- **Actions**: SET NULL, CASCADE, SET DEFAULT

```
CREATE TABLE DEPARTMENT
    ( ... ,
      Mgr_ssn CHAR(9)          NOT NULL        DEFAULT '888665555',
      ... ,
    CONSTRAINT DEPTPK
      PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
      UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
      FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET DEFAULT      ON UPDATE CASCADE);
CREATE TABLE DEPT_LOCATIONS
    ( ... ,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
                  ON DELETE CASCADE          ON UPDATE CASCADE);
```



EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Discussion: Referential Triggered Actions

❑ How to Set Referential Triggered Actions?

- Consider that the EMPLOYEE table's constraint EMPSUPERFK is changed as follows:

```
CREATE TABLE EMPLOYEE
    ( ... ,
      Dno          INT          NOT NULL       DEFAULT 1,
    CONSTRAINT EMPPK
      PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
      FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET NULL          ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
      FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                  ON DELETE SET DEFAULT       ON UPDATE CASCADE);
```

The action for **ON DELETE** is changed to CASCADE

```
CONSTRAINT EMPSUPERFK
  FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
              ON DELETE CASCADE ON UPDATE CASCADE,
```

- Q. What happens when the following command is run?

```
DELETE EMPLOYEE WHERE Lname = 'Borg'
```

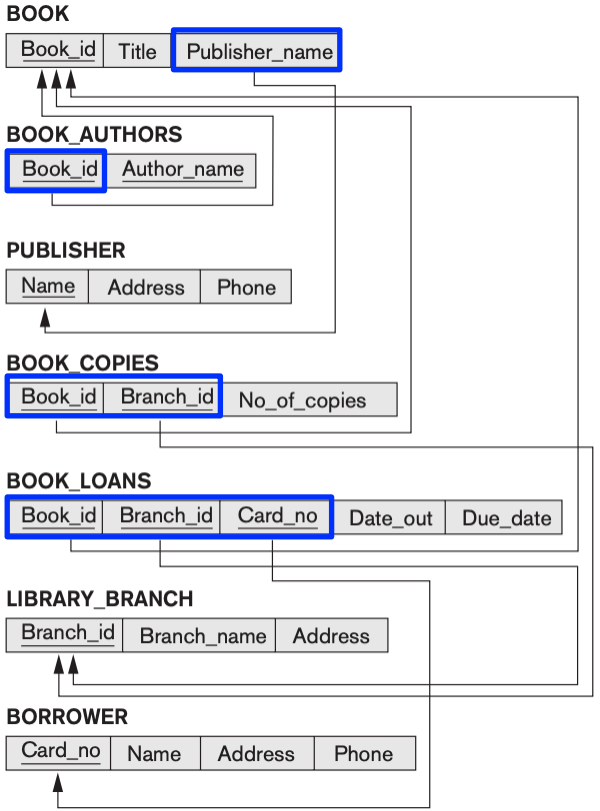
EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Exercises: Specifying Integrity Constraints

❑ Exercise 1

- Consider the LIBRARY relational database schema
- Choose the appropriate action (REJECT, CASCADE, SET NULL, SET DEFAULT) for each referential integrity constraint
  - For the *deletion* of a referenced tuple
  - For the *update* of a primary key attribute value in a referenced tuple

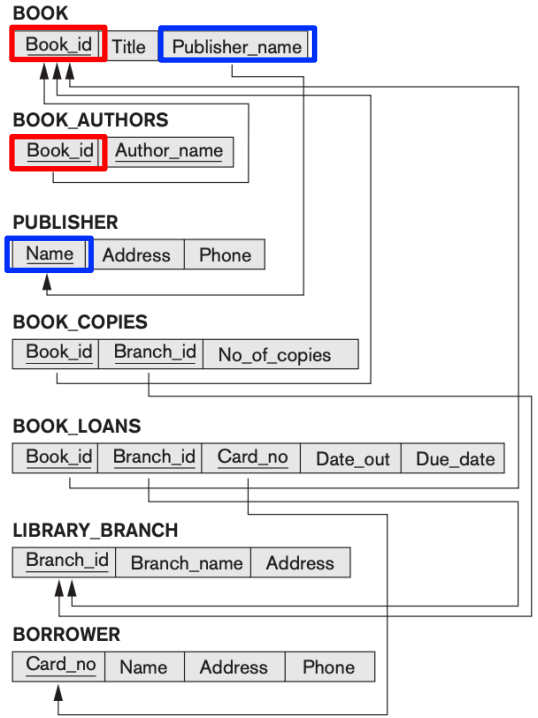


Exercises: Specifying Integrity Constraints

❑ Exercise 1

- BOOK_AUTHORS.Book_id → BOOK.Book_id
  - **ON DELETE** CASCADE
    - Once a book is deleted from a database, the information about the authors of the book to be deleted is no longer necessary
  - **ON UPDATE** CASCADE
    - If the primary key of a book tuple is updated, it should be cascaded to its authors to identify the authors of the book

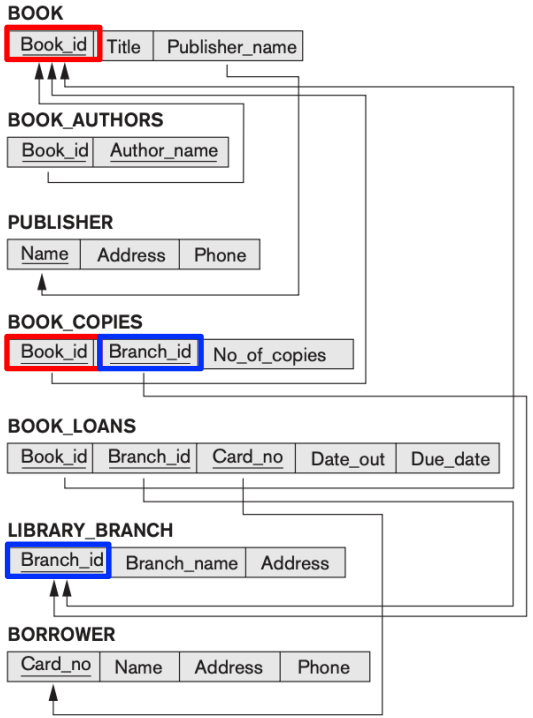
- BOOK.Publisher_name → PUBLISHER.Name
  - **ON DELETE** REJECT
    - If there exist books published by a publisher, we do not allow the publisher to be deleted (a book must have a name of the publisher)
  - **ON UPDATE** CASCADE
    - If the name of a publisher is changed, we want to reflect the change in referencing BOOK tuples



Exercises: Specifying Integrity Constraints

❑ Exercise 1

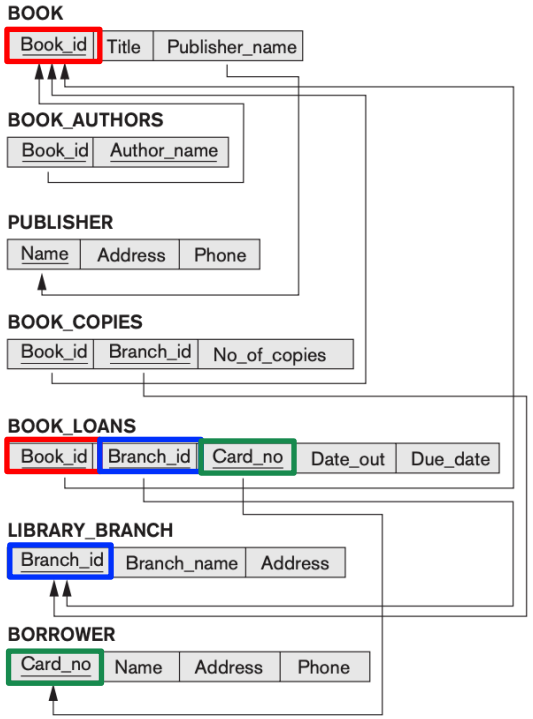
- BOOK_COPIES.Book_id → BOOK.Book_id
  - ❑ **ON DELETE** CASCADE
    - Imagine that a library administrator want to remove a book from the list of all the books in the library
    - Then, the administrator wants to delete the corresponding tuples in BOOK
    - Thus, any copies of this book do not need to be maintained anymore
  - ❑ **ON UPDATE** CASCADE
    - If there is a change in Book_id of a BOOK tuple, the change should be reflected into the referencing tuples (i.e., its copies)
- BOOK_COPIES.Branch_id →LIBRARY_BRANCH.Branch_id
  - ❑ **ON DELETE** CASCADE
    - The deletion of a LIBRARY_BRANCH tuple: actual "closing" of that branch
    - Thus, the book copies do not need to be kept track of anymore
  - ❑ **ON UPDATE** CASCADE
    - If there is a change in Branch_id, the change should be reflected into the referencing tuples in order to correctly keep track of the library branch where the book copies are



Exercises: Specifying Integrity Constraints

❑ Exercise 1

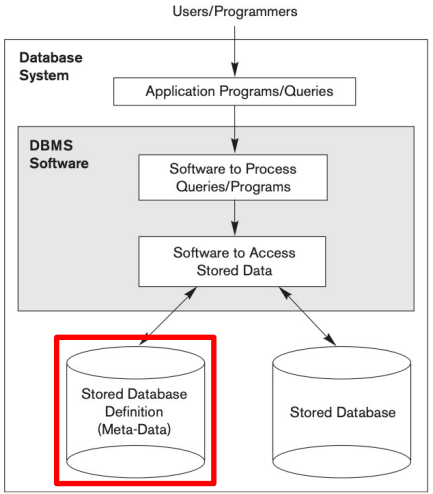
- BOOK_LOANS.Book_id → BOOK.Book_id
  - **ON DELETE**: ?
  - **ON UPDATE**: ?
- BOOK_LOANS.Branch_id →LIBRARY_BRANCH.Branch_id
  - **ON DELETE:** ?
  - **ON UPDATE:** ?
- BOOK_LOANS.Card_no → BORROWER.Card_no
  - **ON DELETE**: ?
  - **ON UPDATE**: ?



Modifying Database Schemas

What is Schema Modification?

- To modify schemas, rather than tuples
  - E.g., adding/dropping tables, attributes, constraints, and other elements
- Schema modification commands:
  - **DROP SCHEMA, DROP TABLE, ALTER TABLE**



The DROP SCHEMA Command

- To drop a whole schema if it is no longer needed

```
DROP SCHEMA COMPANY CASCADE;
```

- Two drop behavior options for **DROP SCHEMA**
  - **CASCADE**: delete all elements of the schema
    - i.e., removing all its tables, domains, and other elements
  - **RESTRICT** ⚠: delete the schema only if it has no elements in the schema
    - The user must individually drop each element in the schema

Modifying Database Schemas (Cont.)

❑ The DROP TABLE Command

- To drop a specific named table

```
DROP TABLE DEPENDENT CASCADE;
```



  - ❑ It not only deletes *all the tuples in the table*, but also removes *the table definition from the catalog*
    - Use **DELETE** command if you want to delete only the tuples but to leave the table definition
- Two drop behavior options for **DROP TABLE**
  - ❑ **CASCADE**: delete all the tuples and its table definition
    - All constrains and other elements that reference the table are dropped automatically from the schema
  - ❑ **RESTRICT** ⚠: delete the table only if it is not referenced in any constraints or by any other elements
    - The user must carefully check other constraints

→ Q. How can we drop *other schema elements* such as domains, data types, or constraints?

Modifying Database Schemas (Cont.)

❑ The ALTER TABLE Command

- To change the definition of a table or other schema elements
  - ❑ Adding or dropping attributes and constraints
  - ❑ Changing a column definition (e.g., data types or constraints)

```
ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN Job VARCHAR(12);

ALTER TABLE COMPANY.EMPLOYEE DROP COLUMN Address CASCADE;
```

```
ALTER TABLE COMPANY.EMPLOYEE
DROP CONSTRAINT EMPSUPERFK CASCADE;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno	Job
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5	
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5	
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4	
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4	
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5	
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5	
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4	
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1	

CASCADE: drop all constraints that reference the column automatically, along with the column

RESTRICT: drop the column only if no schema elements reference the column

Outline

- **Basic SQL: Data Definition Language**
  - Introduction to SQL
  - Creating Schemas and Tables
  - Specifying Constraints
  - Modifying Schemas: DROP and ALTER

- **Basic SQL: Data Manipulation Language**
  - Data Update: INSERT, DELETE, and UPDATE
  - Data Retrieval: SELECT-FROM-WHERE Structure



Update Commands in SQL

❑ Three Commands to Modify the Database

- **INSERT**: add a single tuple to a relation
- **DELETE**: remove tuples from a relation
- **UPDATE**: modify attribute values of one or more selected tuples

```
INSERT INTO   EMPLOYEE
VALUES        ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
              Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );

DELETE FROM   EMPLOYEE
WHERE         Lname = 'Brown';

UPDATE        PROJECT
SET           Plocation = 'Bellaire', Dnum = 5
WHERE         Pnumber = 10;
```

INSERT Command

❑ Usage 1: All Attribute Values

- Specify a relation name and *a list of all attribute values* for the tuple
  - The values should *be listed in the same order* in which the corresponding attributes specified

```
INSERT INTO    EMPLOYEE
VALUES         ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
               Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

INSERT Command (Cont.)

❑ Usage 2: All Explicit Attribute Values

- Specify explicit attribute names and their corresponding values
  - ❑ The values for *all attributes with NOT NULL and no default value* must be included

```
INSERT INTO   EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES        ('Richard', 'Marini', 4, '653298653');
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE TABLE EMPLOYEE
    ( Fname          VARCHAR(15)     NOT NULL,
      Minit          CHAR,
      Lname          VARCHAR(15)     NOT NULL,
      Ssn            CHAR(9)         NOT NULL,
      Bdate          DATE,
      Address        VARCHAR(30),
      Sex            CHAR,
      Salary         DECIMAL(10,2),
      Super_ssn      CHAR(9),
      Dno            INT             NOT NULL,
      PRIMARY KEY (Ssn),
```

INSERT Command: Violating Constraints

❑ Violating Integrity Constraints of INSERT Commands

```
INSERT INTO     EMPLOYEE (Fname, Lname, Ssn, Dno)
VALUES          ('Robert', 'Hatcher', '980760540', 2);
```

No DEPARTMENT tuple with Dnumber=2 exists

→ Violating the referential integrity constraint
Thus, this should be rejected!

```
INSERT INTO     EMPLOYEE (Fname, Lname, Dno)
VALUES          ('Robert', 'Hatcher', 5);
```

No Ssn value is provided

→ Violating NOT NULL constraint
This should be rejected!

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland

DELETE Command

❑ Usage

- **WHERE**: select the tuple(s) to be deleted from **a single relation**
  - ❑ Tuples are explicitly deleted from only one table at a time
  - ❑ The deletion may propagate to tuples in other relations if *referential triggered actions* are specified
    - e.g., **Trigger**: ON DELETE → **Action**: CASCASE
- The number of tuples to be deleted can be different

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DELETE FROM EMPLOYEE **WHERE** Lname = 'Brown';		No tuple is deleted
DELETE FROM EMPLOYEE **WHERE** Ssn = '123456789';		1 tuple is deleted
DELETE FROM EMPLOYEE **WHERE** Dno = 5;		4 tuples are deleted
DELETE FROM EMPLOYEE;		all tuples are deleted

*Missing **WHERE** deletes all tuples!* ⚠

UPDATE Command

❑ Usage

- **WHERE**: select the tuple(s) to be modified from **a single relation**
  - The update may propagate to tuples in other relations if *referential triggered actions* are specified
    - e.g., **Trigger**: ON UPDATE → **Action**: CASCASE
- **SET**: specify the attributes to be modified and their new values

```
UPDATE    PROJECT
SET       Plocation = 'Bellaire', Dnum = 5
WHERE     Pnumber = 10;
```

```
UPDATE    EMPLOYEE
SET       Salary = Salary * 1.1
WHERE     Dno = 5;
```

Several tuples can be modified with a single UPDATE command

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Data Retrieval in SQL

❑ SELECT Command

- SQL has one basic statement for retrieving information from a database
- SELECT-FROM-WHERE block

```
SELECT    <attribute list>
FROM      <table list>
WHERE     <condition>;
```

- **<attribute list>** is a list of attribute names whose values are to be retrieved by the query
  - Projection attributes
- **<table list>** is a list of the relation names required to process the query
- **<condition>** is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query
  - Selection condition and join condition

Data Retrieval in SQL: Examples

❑ Query 1:

- Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'

```
SELECT   Bdate, Address
FROM     EMPLOYEE
WHERE    Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';
```

Projection attributes:
Specified attributes whose values are to be retrieved

Selection condition:
Choose the particular tuples of interest in a table

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Bdate	Address
1965-01-09	731Fondren, Houston, TX

Data Retrieval in SQL: Examples (Cont.)

❑ Query 2:

- Retrieve the name and address of all employees who work for the 'Research' dept.

```
SELECT   Fname, Lname, Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    Dname = 'Research' AND Dnumber = Dno;
```

Projection attributes

Selection condition

Join condition:
Combine two tuples,
one from DEPARTMENT and one from EMPLOYEE

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19



Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Ambiguous Attribute Name

❑ Preventing Ambiguity

- The same name can be used for two (or more) attributes in *different* relations
- We must **qualify** the attribute name with the relation name to prevent ambiguity
  - If multitable query refers to two or more attributes with the same name

```
SELECT   Fname, EMPLOYEE.Name, Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.Name = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

Prefixing the relation name to the attribute name and separating the two by a period(.) to prevent ambiguity

```
SELECT   EMPLOYEE.Fname, EMPLOYEE.LName,
         EMPLOYEE.Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.DName = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

Fully qualified attribute names can be used for better clarity

EMPLOYEE

Fname	Minit	Name	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dnumber
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Name	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Renaming Attributes

❑ Alias (Tuple variable) for Table Names

- Fully qualified attribute names can be used for better clarity
- But they often require repeated typing of long table names…😩

```
SELECT   EMPLOYEE.Fname, EMPLOYEE.LName,
         EMPLOYEE.Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.DName = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

⇨

```
SELECT   E.Fname, E.LName, E.Address
FROM     EMPLOYEE AS E, DEPARTMENT AS D
WHERE    D.DName = 'Research' AND D.Dnumber = E.Dno;
```

Q. Retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT   E.Fname, E.Lname, E.Fname, E.Lname,
FROM     EMPLOYEE AS E
WHERE    E.Super_ssn = E.Ssn;
```

The ambiguity of attribute names also arises when a query refers to the same relation twice…

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Two More Features of SQL

❑ Missing WHERE Clause

- No condition on tuple selection
- Retrieve ***all tuples*** of the relations specified in FROM

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

```
SELECT   Ssn
FROM     EMPLOYEE;
```

E.Fname
123456789
333445555
999887777
987654321
666884444
453453453
987987987
888665555

```
SELECT   Ssn, Dname
FROM     EMPLOYEE, DEPARTMENT;
```

Ssn	Dname
123456789	Research
333445555	Research
999887777	Research
987654321	Research
666884444	Research
453453453	Research
987987987	Research
888665555	Research
123456789	Administration
333445555	Administration
999887777	Administration
987654321	Administration
666884444	Administration
453453453	Administration
987987987	Administration
888665555	Administration
123456789	Headquarters
333445555	Headquarters
999887777	Headquarters
987654321	Headquarters
666884444	Headquarters
453453453	Headquarters
987987987	Headquarters
888665555	Headquarters

All possible tuple combinations

3*8=24

It is extremely important to specify every condition in the WHERE Clause!

if any such condition is overlooked, incorrect and very large relations may result

Two More Features of SQL (Cont.)

❑ Use of the Asterisk (*)

- Retrieve ***all the attributes values*** of the selected tuples
- To avoid inconvenience of specifying all attribute names

We don't have to list the attribute names explicitly in SQL

```
SELECT   *
FROM     EMPLOYEE
WHERE    Dno = 5;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1



Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-09-01	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

Tables as (Multi)Sets

❑ Tables as Multisets in SQL

- SQL treats a table not as a *set* but rather as a ***multiset***
  - ❑ i.e., "duplicate tuples" can exist in a table (or in the result of a query)
- SQL *does not automatically eliminate duplicate tuples* in query results

❑ Why SQL treats Tables as Multisets?

- Duplicate elimination is an expensive operation → why?
  - ❑ E.g., sorting the tuples first and eliminate duplicates whenever new tuples are inserted
- Or sometimes a user may want to see duplicate tuples in the result of a query
- Also, when *aggregate functions* are applied to tuples (→ Week 5)

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

SELECT **ALL** Salary
FROM EMPLOYEE;

Salary
30000
40000
25000
43000
38000
25000
25000
55000

Tables as (Multi)Sets (Cont.)

❑ ALL Keyword

- Does not eliminate duplicate tuples from the result of a query

❑ DISTINCT Keyword

- Eliminate duplicate tuples from the result of an SQL query

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Q1. Retrieve the salary of every employee
Q2. Retrieve all distinct salary values

```
Q1  SELECT  ALL Salary
    FROM    EMPLOYEE;

Q2  SELECT  DISTINCT Salary
    FROM    EMPLOYEE;
```

Q1

Salary
30000
40000
25000
43000
38000
25000
25000
55000

Q2

Salary
30000
40000
25000
43000
38000
55000

Tables as (Multi)Sets (Cont.)

Set Operations on Relations

- Set union, (**UNION**), set difference (**EXCEPT**), set intersection (**INTERSECT**)
- The relations resulting from these set operations are ***sets*** of tuples
  - That is, duplicate tuples are eliminated from the result
- They apply only to ***type-compatible*** relations

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
( SELECT    DISTINCT Pnumber
  FROM      PROJECT, DEPARTMENT, EMPLOYEE
  WHERE     Dnum = Dnumber AND Mgr_ssn = Ssn
            AND      Lname = 'Smith' )
  UNION
( SELECT    DISTINCT Pnumber
  FROM      PROJECT, WORKS_ON, EMPLOYEE
  WHERE     Pnumber = Pno AND Essn = Ssn
            AND      Lname = 'Smith' );
```

Projects in which 'Smith' works as a manager

Projects in which 'Smith' works as a worker

Tables as (Multi)Sets (Cont.)

❑ Multiset Operations

■ UNION ALL, EXCETP ALL, INTERSECT ALL

- ❑ SQL also has corresponding multiset operations, followed by the keyword **ALL**

(a) R

A
a1
a2
a2
a3

S

A
a1
a2
a4
a5

(b) T

A
a1
a1
a2
a2
a2
a3
a4
a5

R(A) **UNION ALL** R(S)

(c) T

A
a2
a3

R(A) **EXCEPT ALL** R(S)

(d) T

A
a1
a2

R(A) **INTERSECT ALL** R(S)

Ordering of Query Results in SQL

❑ ORDER BY Clause

- Ordering resulting tuples of a query by the values of one (more) of the attributes
- The default order is in ascending order of values
  - ❑ **DESC** (Descending order) and **ASC** (ascending order) keywords

Query 15. Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, then first name.

```
Q15:  SELECT    D.Dname, E.Lname, E.Fname, P.Pname
      FROM      DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W,
                PROJECT AS P
      WHERE     D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno =
                P.Pnumber
      ORDER BY  D.Dname, E.Lname, E.Fname;
```

Or

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

Standard Arithmetic Operators

❑ Arithmetic Operators Can be Applied to Numeric Attributes

- Addition (+), Subtraction (-), multiplication (*), division (/)

Query 13. Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise.

This does not change the relation states

```
Q13:  SELECT   E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
      FROM     EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
      WHERE    E.Ssn = W.Essn AND W.Pno = P.Pnumber AND
               P.Pname = 'ProductX';
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

30,000*1.1 = 33,000

25,000*1.1 = 27,500

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

Comparison Operators

❑ BETWEEN Operator

- Specify the range of attribute values in the WHERE clause

Query 14. Retrieve all employees in department 5 whose salary is between $30,000 and $40,000.

```
Q14:  SELECT   *
      FROM     EMPLOYEE
      WHERE    (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

```
((Salary >= 30000) AND (Salary <= 40000))
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Exercises: Data Retrieval in SQL

❑ Exercise 2: Specify the Following Queries in SQL

- Q. Retrieve the names of all employees in department 5 who work more than 10 hours per week on the “ProductX” project

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Exercises: Data Retrieval in SQL

❑ Exercise 3: Specify the Following Queries in SQL

- Q. Retrieve the name and transcript of each senior student (Class=4) majoring in CS. A transcript includes 1) course name, 2) course number, 3) credit hours, 4) semester, 5) year, and 6) grade for each course completed by the student

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Takeaways: Basic SQL

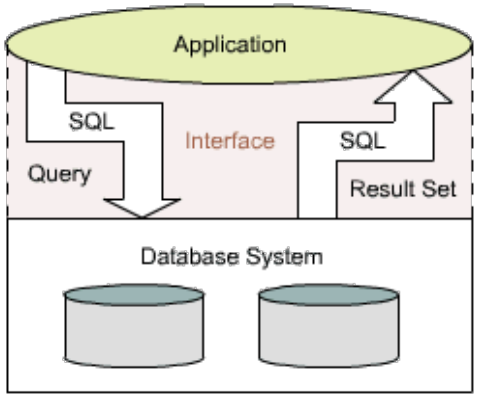
❑ What is a Structured Query Language (SQL)?

- Special-purpose *declarative* programming language
  - ❑ Designed for IBM's first DBMS and originally named **SEQUEL**
  - ❑ *Pronounced by '**S-Q-L**' or '**SEQUEL**'*



❑ Data Definition Language (DDL)

- Providing commands for '**databases and tables**'
  - ❑ Creating/modifying schemas and tables
  - ❑ Specifying constraints



❑ Data Manipulation Language (DML)

- Providing commands for '**tuples**' in tables
  - ❑ Data Update: INSERT, DELETE, and UPDATE
  - ❑ Data Retrieval: SELECT-FROM-WHERE Structure

Q&A:
Basic SQL